



Frankfurt University of Applied Sciences
Master of Engineering
Information Technology

Course:
Agile Development in Cloud Computing Environments

Authors

AMRITA ELIZABETH (1567012)

AYUSH NARESH PARMAR (1565427)

MITHILA MARUTI PRABHU (1567111)

SNEHAMOL SUNNY (1568233)

CONTENTS

ABSTRACT	3
INTRODUCTION.....	4
OBJECTIVE AND SCOPE	4
Objective	4
Scope	4
METHODOLOGY	5
AGILE FRAMEWORK OVERVIEW	5
TEAM STRUCTURE AND ROLES.....	6
TOOLS AND TECHNOLOGIES USED.....	6
SPRINT PLANNING AND EXECUTION.....	7
PLANNING PROCESS.....	7
a) Sprints and Deliverables	7
b) Scrum Meetings	11
c) Application of Agile Principles	11
CHALLENGES AND CRITICAL APPRAISAL	11
Challenges.....	11
Critical Appraisal.....	11
RESULTS AND ACHIEVEMENTS.....	12
WORKFLOW EXPLANATION.....	12
Overview	12
Database Integration	12
APIs for Data Handling.....	13
Role-Based Workflow Responsibilities	13
PROJECT IMPLEMENTATION.....	13
TECHNOLOGY COMPARISON AND CROSS-TEAM EVALUATION.....	20
CONCLUSION	20
REFERENCES.....	21

ABSTRACT

This report documents the development of a workforce planning tool using Scrum and agile development practices. The project aims to build a structured and scalable system to support internal employee assignment processes in a large organization. Core functionalities include staff requests, approvals workflows, skill-based matching, and workload management. Agile practices such as iterative development, sprint planning, regular team discussions, and sprint reviews were applied to support continuous improvement and efficient delivery. The system design is based on predefined use cases and follows a layered project structure with integrated workflow automation for managing approvals and assignments. This report highlights the applied agile principles, sprint planning approach, system implementation structure, and overall project evaluation.

INTRODUCTION

Agile software development has become one of the most widely used approaches for developing modern software systems. Traditional development models typically require all requirements to be defined at the beginning of the project and follow a sequential development process. In practice, this approach often lacks flexibility and makes it difficult to accommodate changes during development. When requirements change at later stages, these models frequently lead to increased costs, delays, and reduced system quality.

Agile methodologies were introduced to address these limitations by supporting iterative development, frequent feedback, and close collaboration between development teams and stakeholders. Agile frameworks such as Scrum emphasize incremental delivery, continuous evaluation, and adaptability throughout the project lifecycle. Research has shown that agile methods are particularly effective in environments where requirements evolve and stakeholder involvement is high, as they improve communication, transparency, and responsiveness to change [1].

In this project, agile development principles are applied to the implementation of a workforce planning tool that supports the internal assignment of employees within an organization. The system involves multiple user roles, including Project Managers, Department Heads, Resource Planners, and Employees, each participating in structured request and approval processes. Due to the dynamic nature of workforce allocation and approval workflows, an agile approach was selected to allow flexible planning, iterative implementation, and continuous improvement. Scrum was used as the primary framework to structure development activities, manage sprints, and support effective collaboration within the team.

OBJECTIVE AND SCOPE

Objective

This project aims to build a user-friendly workforce planning system to support the internal assignment and management of employees within an organization. Agile methods guide the development process, ensuring that the system can adapt to evolving requirements and organizational needs. The main objectives of the project are as follows:

- Project Managers will be able to create and manage staffing requests for new or ongoing projects by defining required skills, workload, and start dates, and by tracking the status of approval and assignment processes.
- Department Heads will be responsible for reviewing staffing requests and approving or rejecting them based on priorities, budget constraints, and resource availability, ensuring transparency in how employees are distributed across projects.
- Resource Planners will be able to search for available employees, match them to project requirements based on skills and current workload, and assign suitable candidates to approved requests while maintaining balanced resource utilization.
- Employees will receive notifications about new assignments and will be able to confirm participation or raise scheduling conflicts, enabling clear communication and reducing assignment-related issues.
- The workforce planning system will support structured approval workflows, automated notifications, and centralized availability management, improving coordination between all involved roles and increasing overall efficiency in workforce allocation.

Scope

The workforce planning system is designed to support internal employee assignment processes within an organization through a centralized platform with defined user roles. The scope of the project focuses on managing staffing requests, approvals, employee matching, and assignment workflows in a structured and transparent manner.

Project Managers can create and manage staffing requests by specifying project details and required skills. These requests are submitted into the system and tracked throughout the approval and assignment lifecycle.

Department Heads review staffing requests and approve or reject them based on organizational priorities and resource availability. The system provides visibility into staffing decisions to support informed approvals.

Resource Planners assign suitable employees to approved requests by evaluating employee skills and availability. The system supports efficient resource allocation and workload balancing across projects. Employees can view assignment notifications, confirm participation, or report scheduling conflicts, ensuring up-to-date availability information.

The project scope also includes workflow-based approval processes, automated notifications, and backend architecture that supports API-based integration. The system is developed as a backend-focused application following agile principles, with emphasis on scalability and maintainability. Advanced analytics and extensive user interface enhancements are considered outside the current scope and may be addressed in future work.

METHODOLOGY

This project used the Scrum agile framework to support a structured and iterative development process. The development was organized into three sprints, each with specific goals and deliverables defined using user stories. Scrum was selected to enable incremental delivery and flexibility in managing evolving requirements during the project lifecycle. Project requirements were identified at a high level and translated into a prioritized backlog. Each sprint followed a consistent cycle consisting of planning, implementation, review, and retrospective phases. Sprint planning focused on selecting backlog items, breaking them into smaller tasks, and estimating effort to support balanced workload distribution. Implementation activities were carried out incrementally, ensuring that each sprint resulted in a working system increment. Regular reviews were conducted to verify that implemented features met functional requirements and aligned with intended workflows. This approach reduced development risk and supported continuous improvement throughout the project [2].

AGILE FRAMEWORK OVERVIEW

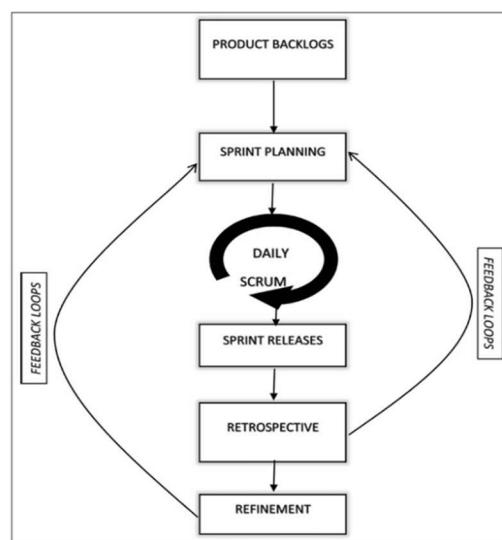


Figure 1: Agile Process Flow

The Scrum framework emphasizes iterative development, transparency, and collaboration [1]. Each sprint began with a planning meeting where tasks were decomposed into smaller, prioritized user stories. Task estimation was performed during planning to manage complexity and ensure achievable sprint goals.

The following Scrum practices were applied during development:

- **Daily Stand-ups:** Short meetings to track progress, discuss completed work, and identify blockers.
- **Sprint Reviews:** Conducted at the end of each sprint to validate completed features against requirements.
- **Sprint Retrospectives:** Used to evaluate team performance and identify improvements.
- **Kanban Board:** Used to track task status across stages such as To-Do, In Progress, Review, and Done.
- **Developers:** Built, tested, and integrated application functionalities across backend services, workflow logic, and frontend components.

Empirical studies show that effective developer collaboration and iterative development practices in agile teams lead to improved software quality, faster delivery, and better overall project outcomes [1], [2].

TEAM STRUCTURE AND ROLES

The project team followed the Scrum framework, which defines three core roles: Product Owner, Scrum Master, and Development Team. These roles were applied in a way suitable for an academic software engineering project while preserving the principles of Scrum.

Product Owner

The Product Owner role was represented by the course instructor. The instructor acted as the primary stakeholder and customer, providing the initial project requirements, defining expectations, and validating outcomes through feedback. The assignment description and milestone feedback served as the product backlog input and acceptance criteria.

Scrum Master

The Scrum Master role was assigned to a team member and rotated across sprints. The Scrum Master was responsible for facilitating Scrum ceremonies such as sprint planning, daily stand-ups, sprint reviews, and retrospectives. Additionally, the Scrum Master supported the team by removing impediments, ensuring adherence to Scrum principles, and maintaining transparency in task progress.

Development Team

All team members collectively formed the Development Team. The Development Team was responsible for designing, implementing, testing, and integrating the system features. Tasks were self-assigned based on availability and expertise, following the pull principle. The team worked collaboratively on backend development, workflow modeling, database integration, and deployment activities.

Role Rotation

To encourage shared responsibility and learning, the Scrum Master role was rotated among team members during different sprints. This approach allowed each member to gain practical experience with Scrum coordination and agile project management.

TOOLS AND TECHNOLOGIES USED

A set of modern development and collaboration tools was used to support agile development, transparency, and reproducibility throughout the project lifecycle.

- Backend & Workflow Engine: Flowable (BPMN-based workflow orchestration)
- Database: PostgreSQL for structured and persistent data storage
- Development Environment: IntelliJ IDEA for backend development
- Containerization: Docker for consistent development and deployment environments
- API Testing: Postman for testing and validating REST endpoints
- Version Control: GitHub for source code management, branching, and collaboration
- Agile Collaboration: Microsoft Teams channels for communication and a Scrum board for sprint planning, backlog management, and task tracking
- Deployment: Render platform for hosting and deploying the application services

These tools supported continuous integration, collaborative development, and iterative delivery in alignment with agile principles.

SPRINT PLANNING AND EXECUTION

PLANNING PROCESS

User stories were derived from the functional requirements of the Workforce Planning system and organized in a prioritized product backlog. Each user story represented a specific interaction within the workforce planning workflow, covering request initiation, approval, employee matching, assignment, and confirmation. Sprint planning sessions were used to select user stories for implementation based on workflow dependencies and technical complexity. Task progress was tracked using a Scrum board in Microsoft Teams with columns for Product Backlog, Sprint Backlog, In Progress, Review, and Done.

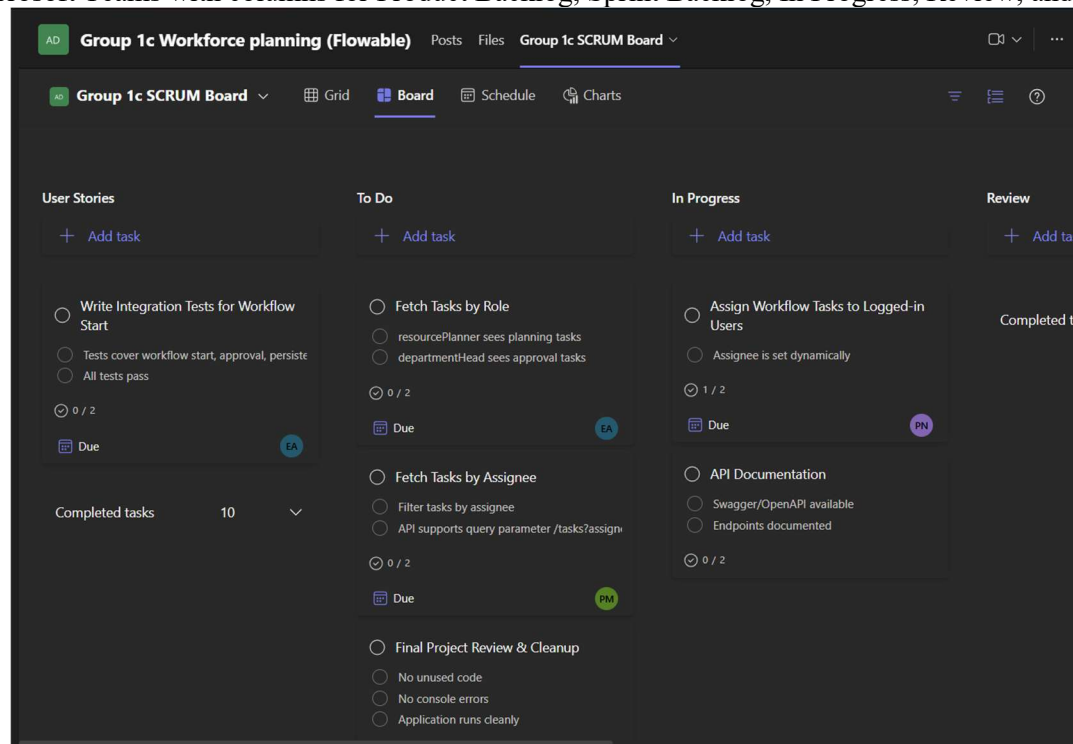


Figure 2: Scrum board showing user stories and task distribution

The Scrum board illustrates how user stories were decomposed into tasks and distributed across team members using a pull-based approach, ensuring transparency, balanced workload, and continuous progress tracking throughout the sprint.

a) Sprints and Deliverables

The project was executed in three sprints, following the official course timeline.

Sprint 1 (31.10.2025 – 28.11.2025)

Focus: Requirement analysis, system design, and workflow definition.

Activities included identifying workforce planning personas, defining core user stories, designing system architecture, and modelling approval and assignment workflows. Initial data entities and relationships were defined during this sprint.

Major User Stories Implemented

- Team Collaboration Environment & Shared Flowable Setup: Establish a common development environment and shared understanding of the Flowable workflow engine.

- Understand Flowable Tool & Create Basic Request Flow: Explore Flowable capabilities by creating a basic request and understanding UI components.
- Create Flowable Workflow / Architectural Diagram: Design the end-to-end workforce planning workflow including approval and rejection paths.
- Backend Project Setup & BPMN Deployment: Initialize backend project structure and deploy initial BPMN workflows.
- Database Setup & Entity Creation (Staffing Request, Employee, Project): Define core entities required to persist workflow and application state.
- Create Project & Employee Repositories: Set up repositories to manage project and employee data.
- Seed Sample Data: Populate the system with sample employees, skills, and projects to support early testing.
- Local Setup and Application Execution: Run the workforce planning application locally and access existing login and dashboard pages.

Additional exploratory and setup-related tasks were performed during this sprint to support system understanding and environment stability.

Deliverables:

- System architecture and workflow diagrams
- Initial BPMN workflow models
- Database schema and core entity definitions
- Working local application setup

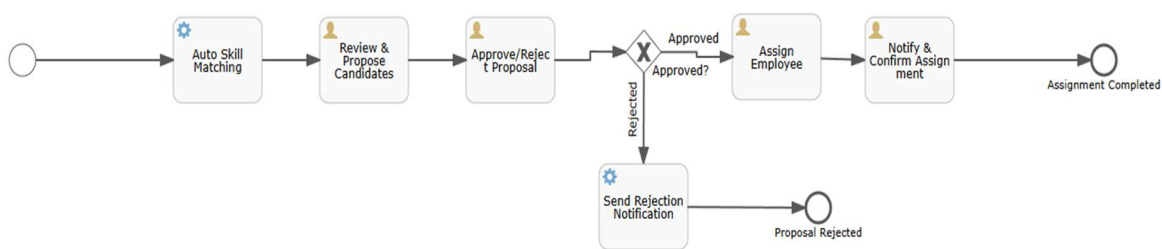


Figure 3: Basic flowable workflow design

Sprint 2 (28.11.2025 – 19.12.2025)

Focus: Backend development and workflow execution.

This sprint focused on implementing REST APIs, integrating workflow logic, and enforcing role-based access control. Backend services were developed incrementally and version-controlled using GitHub.

Major User Stories Implemented

- Project Approval & Rejection Workflow Integration: Implement approval and rejection logic within the workflow engine.
- Start Workflow via REST API & Track Processes: Enable workflows to be started and monitored using REST APIs.
- Implement Assignment Creation Delegate: Automatically create assignments when workflows reach completion.
- Persist Assignment When Workflow Ends: Ensure assignment data is stored reliably upon workflow termination.
- Fetch Tasks by Assignee / Role: Retrieve workflow tasks dynamically based on logged-in user role.
- Approve and Reject Workflow Tasks via API: Allow approvers to act on workflow tasks through backend endpoints.
- Approval History Robustness & Decision Visualization: Track and clearly display approved and rejected decisions.

- Resource Planner Notification & Workflow Visibility: Notify planners and provide real-time visibility into workflow state.

Several additional backend fixes, debugging activities, and workflow refinements were carried out but are not exhaustively listed here.

Deliverables:

- Backend REST APIs
- REST APIs for workflow initiation, approval, and task handling
- Executable BPMN workflows integrated with backend logic
- Assignment creation and persistence logic
- Version-controlled backend codebase

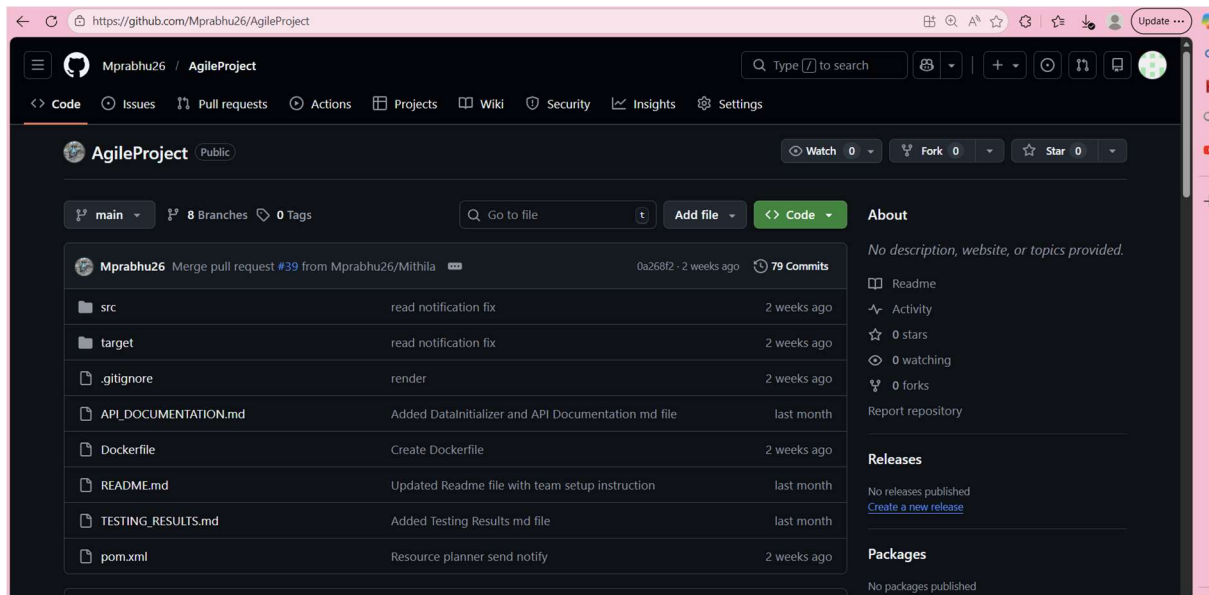


Figure 4: GitHub repository overview showing project structure and version control setup

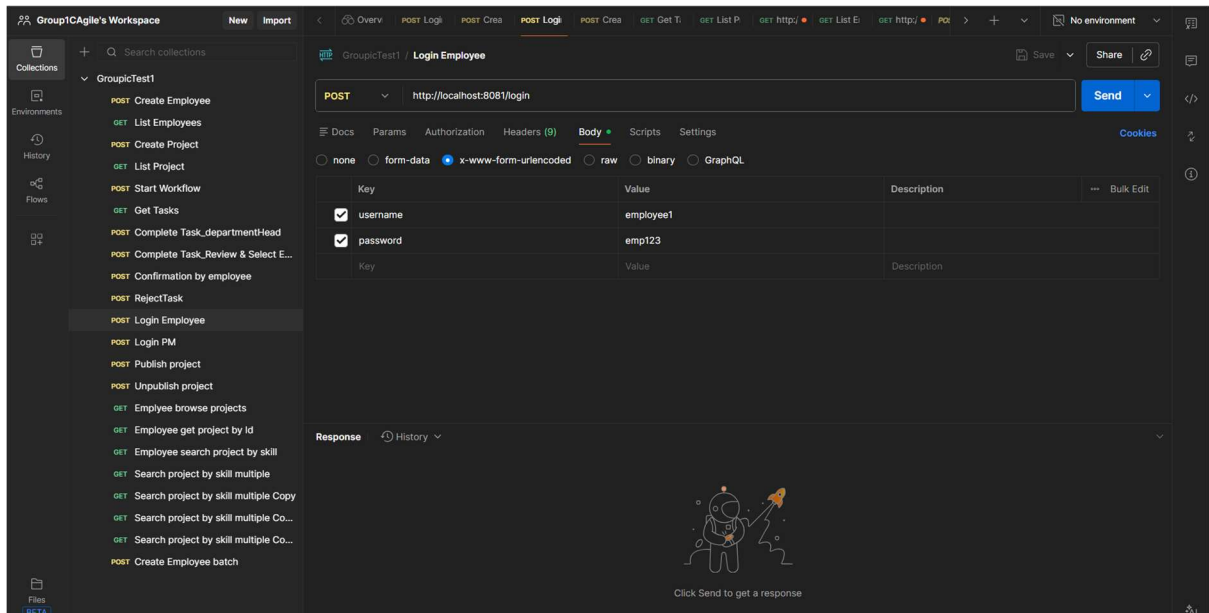


Figure 5: REST API execution for workforce planning workflow using Postman

Sprint 3 (19.12.2025 – 16.01.2026)

Focus: Frontend integration, end-to-end validation, and final deployment.

Frontend components were integrated with backend services and workflows to enable complete request-to-assignment functionality. Role-specific user interfaces were implemented and validated.

Major User Stories Implemented

- Resource Planning Overview Dashboard: Provide planners with a consolidated view of staffing and assignments.
- Project Staffing & Department Head Dashboards: Enable role-specific dashboards with approval and status visibility.
- Automated Skill-Based Project Search & Gap Analysis: Suggest suitable employees based on skills and availability.
- Published Project Visibility & Project Lifecycle Management: Manage project publishing, status control, and visibility.
- Automated Workflow Notifications & PM Alerts: Notify users about approvals, assignments, and workflow events.
- Assignment Validation and Status Updates: Validate assignments and update project and employee states.
- End-to-End Workflow Execution & Testing: Validate complete request-to-assignment flow including rejection paths.
- Cloud Deployment & Final Application Access: Deploy the workforce planning application and verify accessibility.

Additional UI refinements, testing tasks, and cleanup activities were performed during this sprint to ensure stability and usability.

Deliverables

- Role-based dashboards and user interfaces
- Fully integrated end-to-end workforce planning workflow
- Automated notifications and assignment validation
- Deployed and accessible application

Sprint 4 (16.01.2026 – Project Submission)

Focus: System validation, cross-group comparison, gap fixing, and final documentation.

During this final phase, the completed workforce planning system was thoroughly validated to ensure functional correctness, stability, and completeness. The focus shifted from feature development to testing, evaluation, refinement, and reporting. Feedback from other project groups was incorporated to analyze strengths and limitations of the implemented solution and to address any missing or weak functionalities.

Major User Stories Implemented

- Comprehensive Endpoint & Functional Testing: Validate all REST endpoints, workflows, and user interface functionalities using structured testing and supporting screenshots.
- Cross-Group Data Sharing & Evaluation: Share sample data, API structures, and workflow behavior with other project groups to enable comparative analysis.
- Peer Feedback Collection (Pros & Cons Analysis): Collect and analyze advantages and limitations of the implemented solution based on feedback from other teams.
- Identification and Fixing of Missing Functionalities: Review the system against original user stories and use cases, identify functional gaps, and apply corrective fixes.
- Final Report Preparation and Documentation: Consolidate sprint outcomes, testing evidence, and evaluation results into the final software project report.

Additional Activities

- End-to-end workflow verification including approval, rejection, and rollback scenarios.
- Validation of role-based dashboards and access control behavior.
- Minor UI refinements and bug fixes to improve usability and consistency.

- Alignment of documentation with agile reporting requirements and submission guidelines.
- Deliverables

- Fully tested and validated workforce planning application
- Documented comparison and evaluation based on peer group feedback
- Refined system with resolved functional gaps
- Finalized software project report and submission package

b) Scrum Meetings

Scrum meetings were conducted throughout the project, including sprint planning, sprint reviews, and sprint retrospectives. These meetings supported progress tracking, validation of completed work, and continuous improvement.

c) Application of Agile Principles

Agile principles were applied through shared Scrum roles, backlog-driven sprint planning, and continuous collaboration. Tasks were prioritized and selected using push and pull principles, with complexity estimated based on workflow dependencies and integration effort. Regular sprint reviews enabled feedback-driven refinement of workflows and user interfaces. Overall, Scrum roles and agile practices supported transparency, coordination, and incremental progress throughout the project.

CHALLENGES AND CRITICAL APPRAISAL

Challenges

- **Interpreting High-Level Workforce Planning Requirements:** The project requirements described workforce planning only at a conceptual level. Translating these abstract descriptions into concrete workflow steps, role responsibilities, and system behavior required several internal discussions and iterative refinements.
- **Complexity of Workflow Modeling and State Management:** Designing a BPMN-based workflow that correctly handled approvals, rejections, assignments, and confirmations across multiple roles introduced complexity. Ensuring correct task sequencing and workflow state transitions required repeated testing and adjustments.
- **Role-Based Visibility and Workflow Coordination:** Ensuring that workflow tasks, dashboards, and notifications were visible only to the appropriate user roles (Project Manager, Resource Planner, Department Head, Employee) required careful coordination between workflow logic and application-level access control.

Critical Appraisal

- **Applying Scrum:** Scrum practices such as sprint planning, sprint reviews, and retrospectives provided a clear structure for managing development activities and adapting to evolving workflow interpretations.
- **Push-Pull Principle:** The push-pull task allocation approach enabled effective workload management. High-priority workflow tasks were identified during sprint planning, while team members pulled tasks aligned with their expertise.
- **Task Complexity Measurement:** Task complexity was assessed during sprint planning by considering workflow dependencies, integration effort, and potential implementation risks, which supported balanced task distribution across sprints.
- **Customer Collaboration:** Regular demonstrations during sprint reviews allowed early validation of workflow behavior and ensured alignment with expected workforce planning processes.
- **Opinion on Agile Principles:** The iterative agile approach was effective in refining complex workflows. However, repeated refinement cycles occasionally increased planning overhead, particularly when adjusting workflow logic.

- **Roles of Product Owner and Scrum Master:** The Product Owner ensured prioritization of workflow-critical features, while the Scrum Master facilitated coordination and helped resolve impediments. Rotating these roles encouraged shared responsibility and team cohesion.
-

RESULTS AND ACHIEVEMENTS

The project successfully met its objectives by delivering a structured and workflow-driven workforce planning application. The key results and achievements are as follows:

Functional Role-Based Application:

- A scalable and user-friendly workforce planning application was developed to support Project Managers, Resource Planners, Department Heads, and Employees.
- Role-based dashboards and views enabled clear task visibility, approvals, and assignment tracking, supporting effective decision-making.

Process Optimization:

- Regular sprint retrospectives contributed to improved task prioritization, better workflow refinement, and efficient handling of evolving requirements.
- Workflow-related challenges were addressed iteratively, ensuring smooth execution of approval and assignment processes.

Improved Workflow Execution:

- The adoption of Scrum practices enabled effective sprint planning, structured task tracking, and collaborative development.
- Workflow automation using a BPMN-based engine reduced manual coordination and improved consistency in workforce planning processes.

On-Time Delivery:

- All major planned features, including workflow automation, role-based dashboards, and assignment management, were delivered within the project timeline, demonstrating effective time and resource management.

WORKFLOW EXPLANATION

Overview

The Workforce Planning System is implemented as a BPMN-based, workflow-driven application that coordinates staffing activities across multiple organizational roles. The workflow manages the complete lifecycle of a staffing request, from project creation and approval to employee assignment, confirmation, and completion.

The process starts when a Project Manager creates and publishes a project with required roles and skills. The request is reviewed by the Department Head and once approved, is forwarded to the Resource Planner for employee allocation based on skill match and availability. Assigned employees must confirm their participation before the workflow proceeds. System tasks automatically handle status updates, notifications, validations, and exception cases such as rejections or rollbacks.

Using BPMN ensures a transparent, auditable, and adaptable process, providing clear visualization of decision points, task sequencing, parallel flows, and exception handling paths.

Database Integration

- The backend is connected to a PostgreSQL database that stores core workforce planning data, including users, employees, projects, skills, applications, assignments, and notifications
- Business entities such as workforce requests, employee applications, and final assignments persisted in maintaining consistency between workflow execution and application state.
- While Flowable manages workflow execution using its internal engine tables, the application maintains separate domain tables to ensure traceability from project creation to final resource assignment.

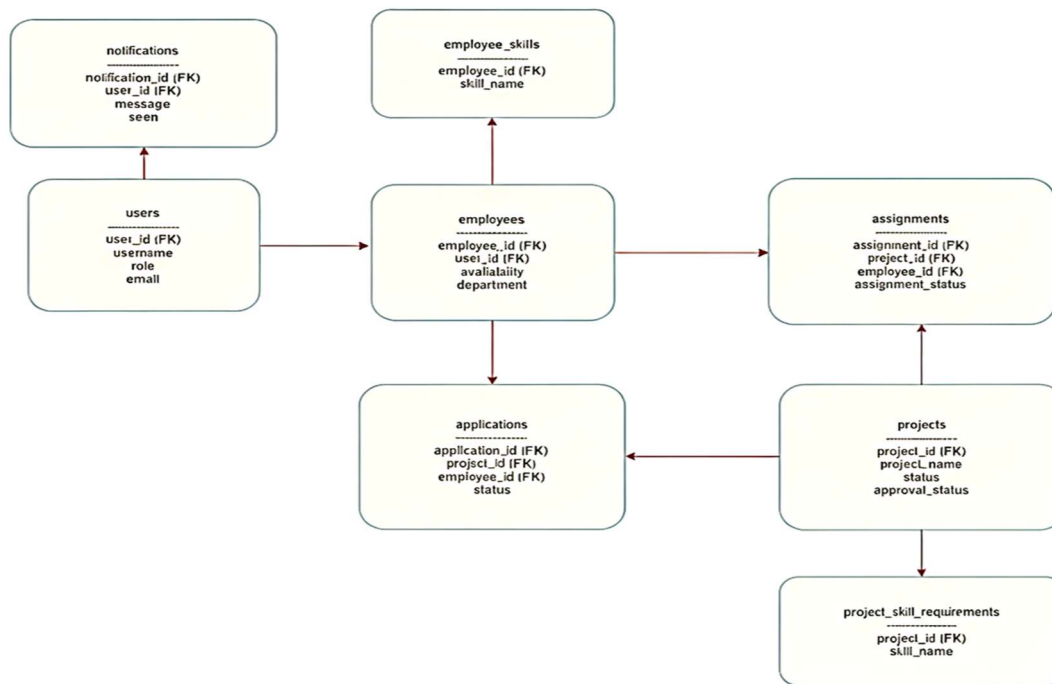


Figure 6: Entity Relationship Diagram (ERD) of the Workforce Planning System

APIs for Data Handling

- RESTful APIs were implemented to connect the frontend, backend services, and the Flowable workflow engine.
- These APIs support key operations such as creating workforce requests, starting workflow instances, retrieving tasks assigned to users, approving or rejecting requests, and persisting assignment results.
- API-based communication ensures loose coupling between system components and allows workflows to be executed and monitored dynamically.

Role-Based Workflow Responsibilities

The workflow explicitly separates responsibilities across defined user roles to ensure clarity, accountability, and controlled access.

- **Project Manager:** Initiates the workflow by creating and publishing project requests and monitors progress through notifications and status updates.
- **Department Head:** Reviews staffing requests and makes approval or rejection decisions, directly influencing workflow transitions.
- **Resource Planner:** Allocates employees to approved projects, manages skill matching and availability checks, and advances the workflow toward assignment confirmation.
- **Employee:** Confirms or rejects assigned projects, enabling the workflow to either complete successfully or revert to corrective steps.
- **System:** Executes automated tasks such as skill gap analysis, status transitions, notification delivery, and data persistence without manual intervention.

This clear role separation ensures that each workflow step is executed by the appropriate stakeholder while maintaining process integrity and traceability.

PROJECT IMPLEMENTATION

HOME PAGE

- In home page you can either login to your account or create a new profile.
- Users can login as Project Manager, Department Head, Resource Planner or Employee. This directs you to the corresponding dashboard.

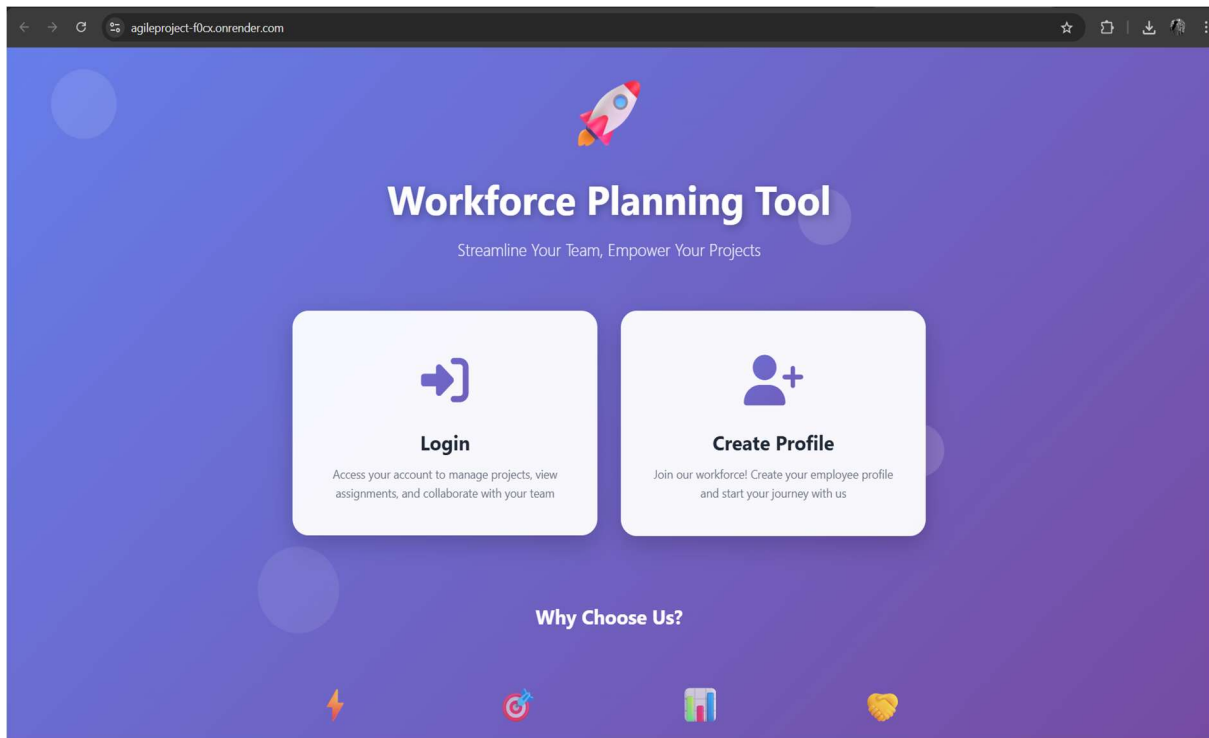


Figure 7: Home page

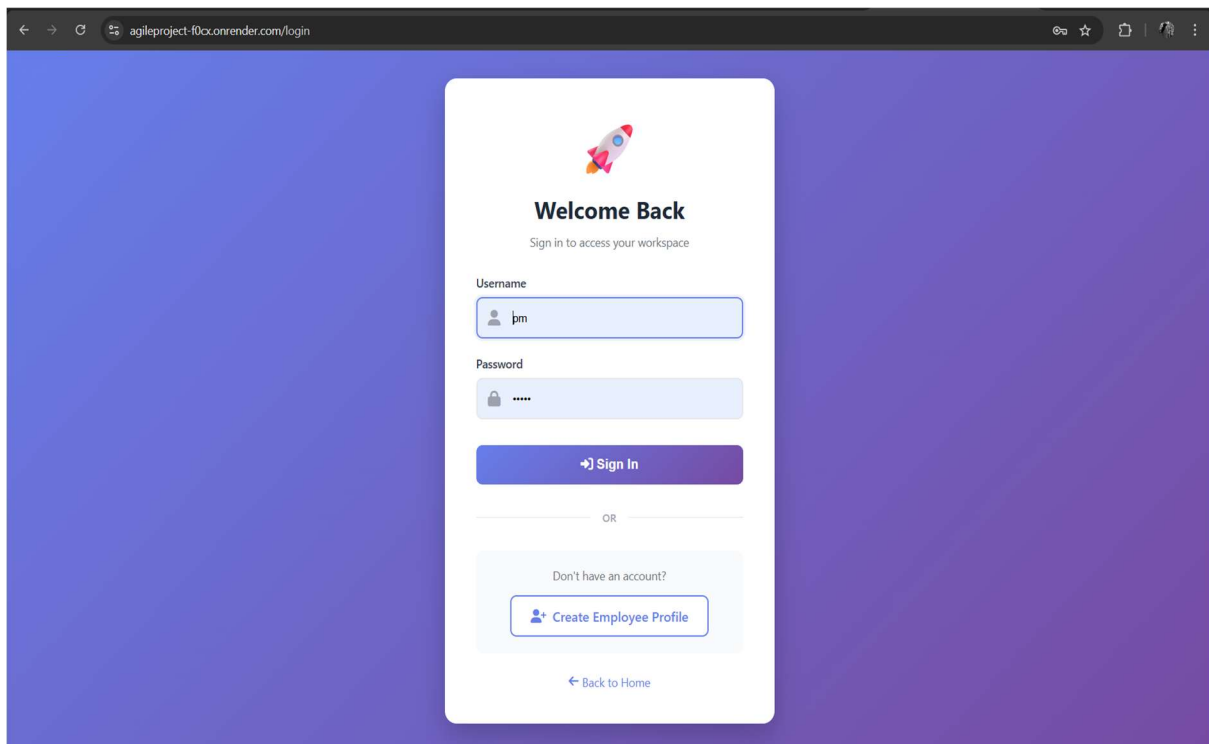


Figure 8: Login Page

Join Our Team
Create your employee profile

Full Name *
e.g. John Doe

Email Address *
pm
Your username will be: pm

Password *
[password field]

Confirm Password *
Re-enter password

Department *
-- Select Department --

Job Role *
e.g. Senior Developer, Designer

Skills *
e.g. Java, Python, React, Project Management
Separate multiple skills with commas

☒ Available for project assignments

Create Account

Already have an account? [Login here](#)

Figure 9: Sign up for new account

PROJECT MANAGER

- The Project Manager dashboard provides an overview of all projects along with their status and progress.
- The Project Manager can create new projects by specifying key details such as required skills, number of employees needed, project duration, and a detailed project description.
- After creating a project, the Project Manager can publish it, making it available for further approval and staffing workflows.
- The Project Manager can review assigned employees and, if required, remove employees from a project to manage staffing effectively.

My Projects Dashboard pm

+ New Project Logout

PROJECT NAME	STATUS	VISIBILITY	DATES	BUDGET	TEAM SIZE	SKILLS REQUIRED	CREATED	ACTIONS
Banking Project Core banking system modernization	APPROVED	Published Jan 27, 2026	Jan 15, 2025 Jun 30, 2025	€1,500,000.00	5	Java 2, Python 2, React 1	pm Jan 27, 2026	Unpublish, View
IoT Platform IoT analytics and monitoring platform	PENDING	Draft	Feb 01, 2025 Sep 30, 2025	€2,200,000.00	4	Java 2, Python 1, React 1	pm Jan 27, 2026	Publish, View
Mobile Banking App iOS and Android banking application	APPROVED	Published Jan 27, 2026	Mar 01, 2025 Aug 31, 2025	€800,000.00	3	React Native 2, Java 1	pm Jan 27, 2026	Unpublish, View

Management Tips
Publish projects to make them visible to resource planners. Only Pending or Approved projects can be edited or deleted.

Figure 10: Dashboard for project manager

Create New Project pm

[← Back to Projects](#) [Logout](#)

New Project Details

Fill in the form below to create a new workforce planning project. All fields marked with * are required.

Basic Information

H Project Name *
Enter project name

Team Size Required *
Number of employees

Start Date *
01/26/2026

End Date *
02/26/2026

Project Budget (€) *
Enter project budget

Project Description
Describe the project goals, objectives, deliverables, and scope...

Required Skills & Resources

Specify the skills and number of people required for each skill. At least one skill is required.

Primary Skill (Required)
e.g., Java, React, Python, C

Secondary Skill (Optional)
e.g., Database, Testing, UI

Additional Skill (Optional)
e.g., Security, Cloud, Mobile

Number Required
Count

[Cancel](#) [Create Project](#)

Please review all information before submitting

Figure 11: Creating a new project from project manager dashboard

DEPARTMENT HEAD

- Once the Project Manager publishes a project, the workflow is initiated and the project appears in the Department Head dashboard.
- The Department Head can review the published project details and choose to approve or reject the request.
- The dashboard provides a clear overview of pending approvals and their current status.
- External Search Requests can also be reviewed separately through the dashboard for further action.

Workforce Planning head [Department Head](#) [Logout](#)

Department Head Dashboard

Review project requests, approve staffing, and monitor department resources.

Published Projects Pending

1

Published Projects

External Search Requests

0

Awaiting review

Total Pending Approved

1

Total Requests

Published Projects Awaiting Approval 1

Projects published by Project Managers - require your approval

App security

Published by: pm • Jan 26, 2026 21:05

AWAITING APPROVAL

[Review](#) [Approve](#) [Reject](#)

External Search Requests

Review requests to search for external candidates.

[View Requests](#)

Figure 12: Department head dashboard

RESOURCE PLANNER

- After Department Head approval, the project appears in the Resource Planner dashboard for staffing activities.
- The Resource Planner reviews employee availability and matches candidates based on required skills.
- Identified skill gaps are clearly highlighted in the UI when required skills cannot be fulfilled internally.
- The Resource Planner can notify the Project Manager and the Department Head about missing skills to initiate external hiring or further approval actions.

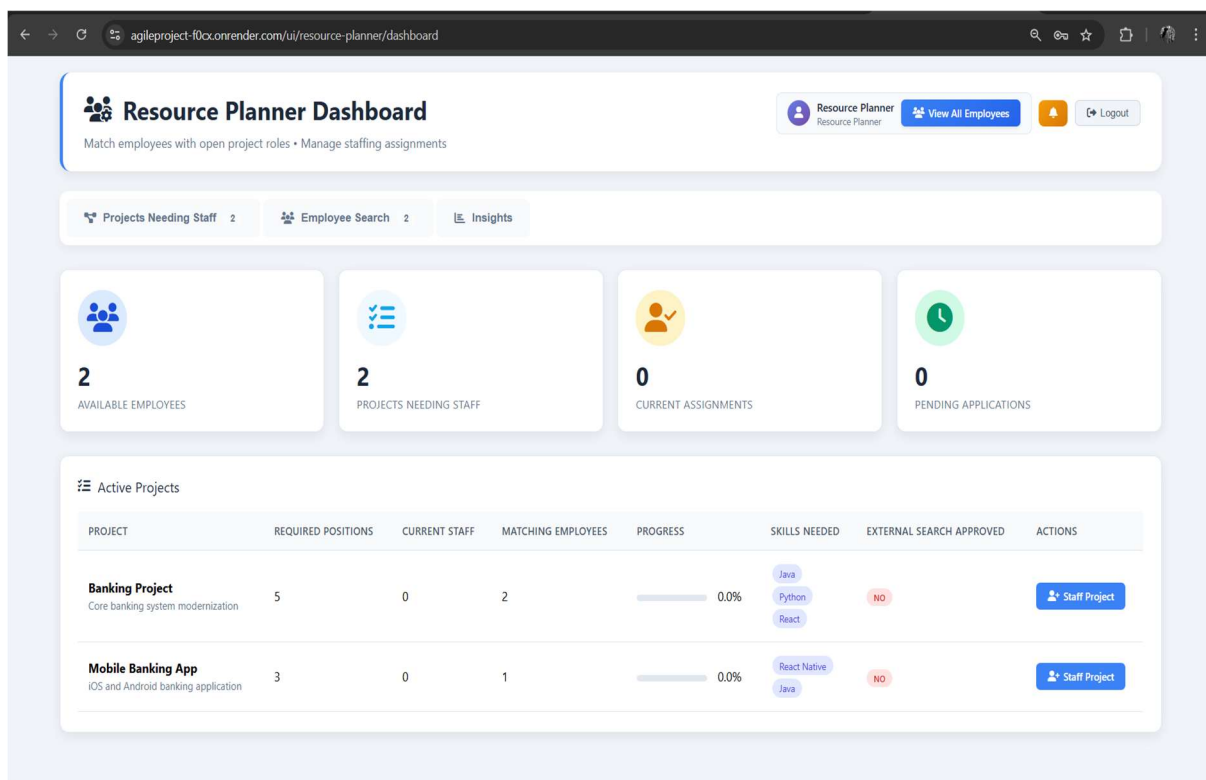


Figure 13:Resource planner dashboard

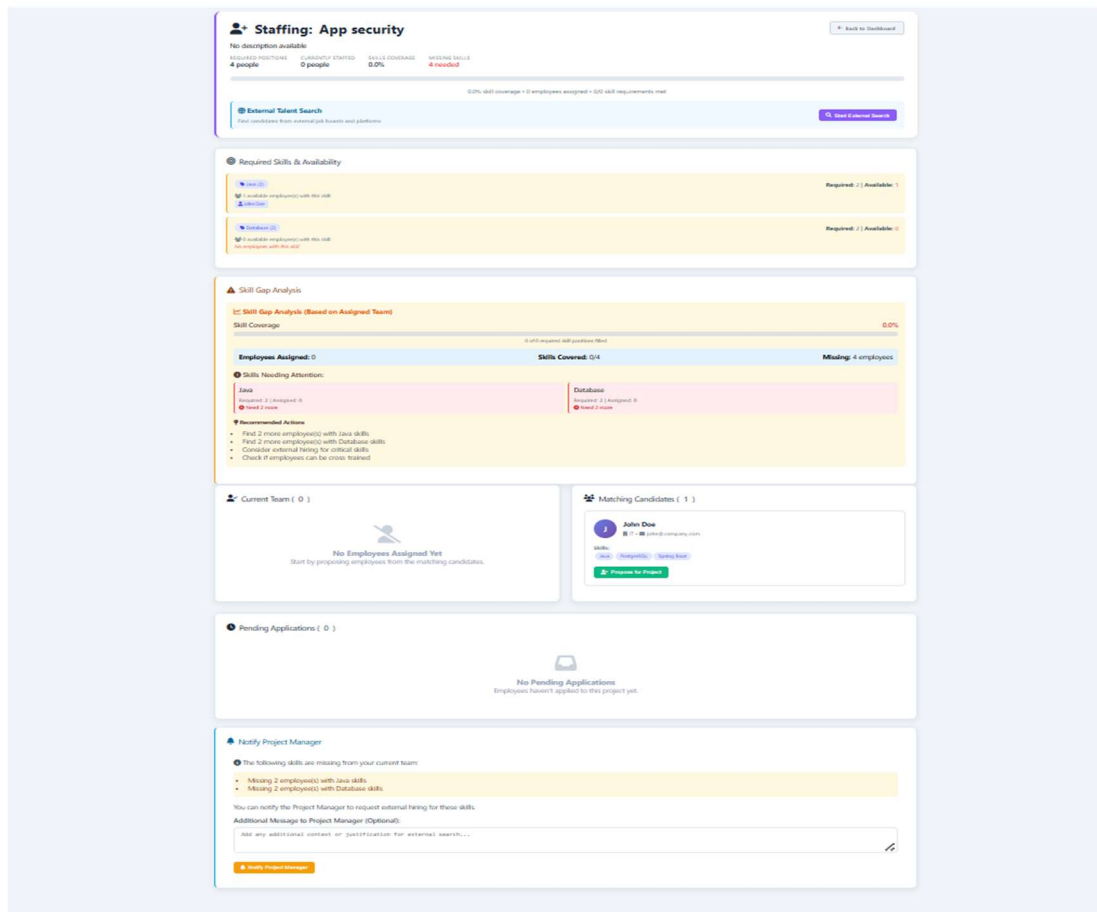


Figure 14: Assigning staff functionality in resource manager dashboard

EMPLOYEE

- Once a project is approved, it becomes visible in the employee dashboard, where employees can browse projects and apply if their skills match the project requirements.
- Employees can search and filter available projects based on required skills to identify suitable opportunities.
- When an employee is proposed and assigned to a project by the Resource Planner, a notification is displayed and the assignment appears in the My Assignments page, where the employee can confirm or reject it.
- Upon accepting a project assignment, the employee's availability status is updated, making them unavailable for other projects.
- All projects an employee has applied to are tracked and displayed in the My Applications section for transparency and follow-up.
- Employees can view and edit their profile information, including skills and availability.

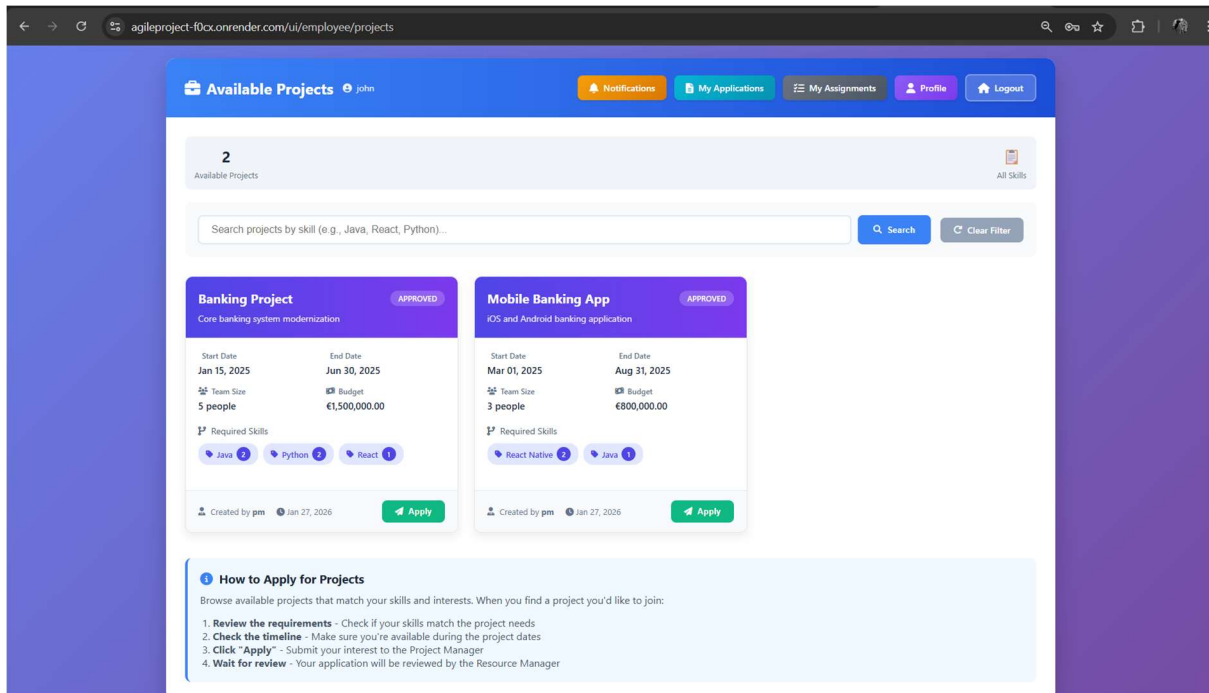


Figure 15:Employee dashboard

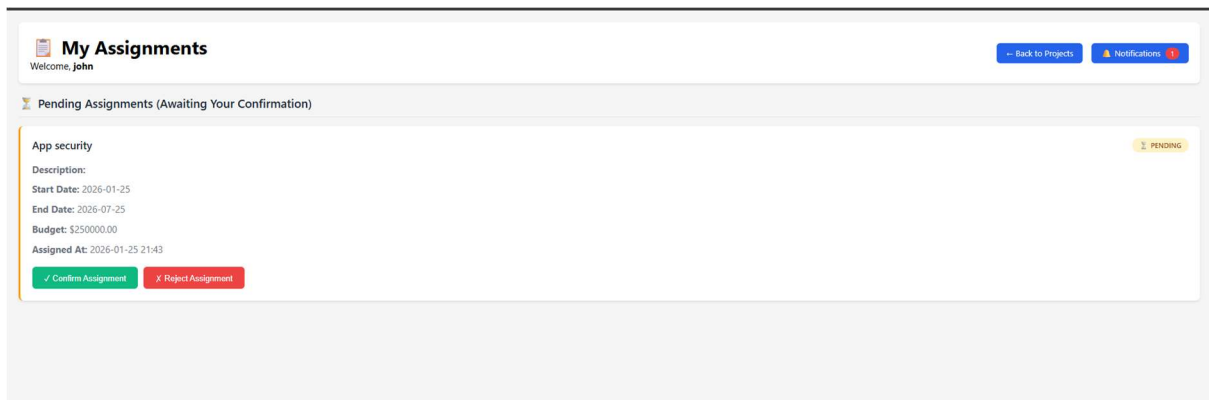


Figure 16:My Assignments page with the project assigned to the employee

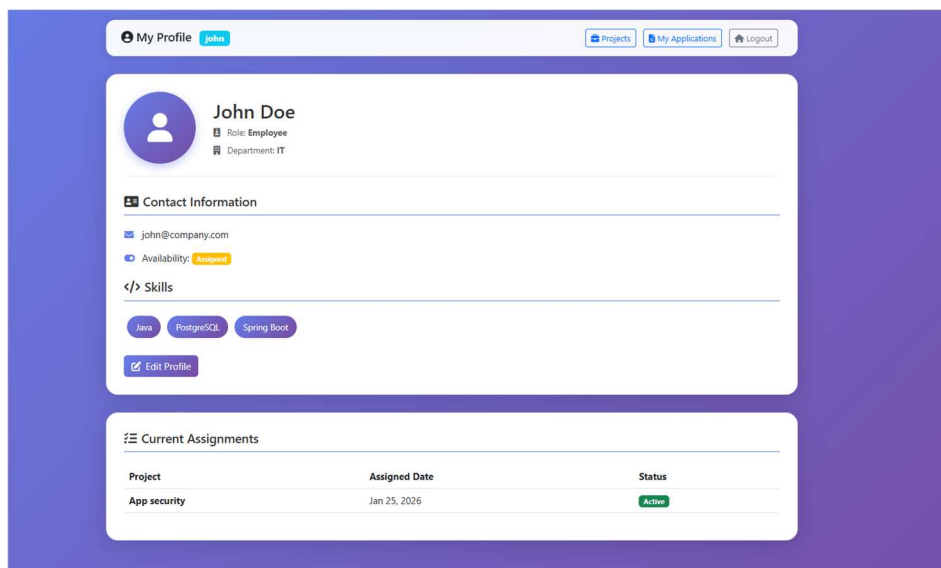


Figure 17:Employee profile

TECHNOLOGY COMPARISON AND CROSS-TEAM EVALUATION

In the final phase of the project, a comparative evaluation was carried out across four team implementations. This comparison highlights key trade-offs between flexibility, development effort, workflow support, and usability.

Manual Workflow Logic using React and Spring Boot

This implementation is a full-stack solution using React, Spring Boot, and Render for deployment. This approach offered full architectural control and high flexibility, allowing custom APIs, data models, and frontend components to be designed precisely according to requirements. The stack reflects widely used industry practices and supports scalability.

However, workflow behavior such as approvals, state transitions, and rollbacks had to be implemented manually in code. This increased development effort and complexity, particularly for process-heavy logic. In addition, the lack of a visual process model made workflows harder to understand for non-technical stakeholders.

Camunda-based Workflow Solution

Camunda was used as a BPM engine alongside Spring Boot and React. This approach combined structured BPMN-based workflows with a modular backend and frontend architecture. Camunda provided better process transparency compared to a fully manual implementation.

At the same time, the solution required considerable setup, configuration, and operational effort. Workflow versioning, deployment management, and infrastructure monitoring added complexity, making the stack more demanding in terms of expertise and maintenance.

Appian Low-Code Platform

The Appian-based solution is based on faster development through a low-code approach. Workflow modeling, UI creation, role management, and notifications could be integrated which helped in prototyping and early feedback. Built-in versioning and rollback feature reduced deployment risks.

However, the Community Edition imposed limitations on data modeling and advanced customization. Integration with external systems and fine-grained UI control were more restricted compared to developer-centric platforms, which reduced flexibility for complex use cases.

Flowable-based Workflow Solution (This Project)

This project used Flowable as an embedded workflow engine within a Spring Boot backend. The BPMN-based process modeling enabled clear representation of approvals, assignments, and role-based decisions. Flowable's REST APIs and Java delegates allowed workflows to be tightly integrated with custom business logic while keeping backend control.

Some challenges were observed in monitoring and debugging running workflows, as advanced visual tooling is limited compared to commercial platforms. Nevertheless, the solution remained lightweight, flexible, and well-suited for integration into a custom application.

Comparative Summary

Each approach presents different trade-offs. Manual implementations offer high flexibility but require greater development effort. Low-code platforms enable rapid development but limit customization. Workflow engines such as Camunda and Flowable balance structured process modeling with flexibility.

CONCLUSION

This project demonstrated how agile development principles can be effectively applied to the design and implementation of a workflow-driven workforce planning system. By adopting Scrum methodologies, the project development was structured into manageable iterations to improve the technical and functional aspects of the system. The use of sprints, reviews, and retrospectives provided a clear rhythm for progress while allowing flexibility to adapt to evolving interpretations of the requirements.

From a technical perspective, the integration of a BPMN-based workflow engine proved well suited for modeling complex approval and assignment processes involving multiple roles. The workflow-centric approach helped ensure transparency, consistency, and traceability across project creation, approval, resource allocation, and employee confirmation. While this approach introduced additional complexity compared to simpler CRUD-based solutions, it enabled a clear separation between process logic and application code, which improved maintainability and extensibility.

The final evaluation phase highlighted the importance of validation and reflection in agile projects. Systematic testing, cross-team comparison, and peer feedback revealed both strengths and limitations of the implemented solution. Comparing different technological approaches across teams showed that no single solution is universally optimal; rather, each involves trade-offs between flexibility, development effort, usability, and process control. In this context, the selected architecture achieved a balanced compromise between structured workflow modeling and backend flexibility.

Overall, the project achieved its core objectives by delivering a functional, role-based workforce planning application supported by an end-to-end workflow. Beyond the technical outcome, the project provided valuable practical experience in applying agile methods, collaborating within a team, and critically evaluating design decisions. These insights reinforce the relevance of agile development practices for managing complex, real-world software systems where requirements, constraints, and solutions evolve over time.

REFERENCES

- [1] T. Dybå and T. Dingsøyr, “Empirical studies of agile software development: A systematic review,” *Information and Software Technology*, vol. 50, no. 9–10, pp. 833–859, Aug. 2008.
- [2] N. Anifa, A. A. Rahman, and M. S. Hossain, “A Systematic Literature Review on Agile Approach in Software Development,” 2024.